A Header-Only, Attention-Aware Memory Estimator for GGUF-Quantized LLMs on Edge Devices

Venumadhav Kalluru
Independent Researcher
Cumming, GA, USA
greenkal2026@gmail.com
ORCID: 0009-0006-8250-9899

***Abstract*—People who run quantized language models locally often discover that a checkpoint does not fit only after downloading several gigabytes of weights. We ask how much of that answer the GGUF header can give for llama.cpp on a CPU host. The estimator reads only the architecture fields, which end within the first 1.3 KB of each tested file, and adds three terms: weight bytes from the file size, a key–value (KV) cache sized from the effective KV-head count, the stored key and value head widths and the block size of the cache type, and a runtime constant calibrated once per backend. On 36 configurations of six Q4_K_M models (0.5–3.8 B parameters), the mean absolute percentage error (MAPE) is 2.25% under leave-one-model-out calibration, and no configuration is under-predicted by more than 3.5%. Using total instead of KV heads raises MAPE to 31% on the grouped-query models. A second run that fills the context shows that peak memory at 8K tokens exceeds the short-prompt peak by up to 84%, because the attention buffers become resident. An attention-workspace term derived from the same header brings the filled-context error to 1.71%. GPUStack's gguf-parser, run on the same grid, over-predicts by 63% on average. All measurements are CPU-only.**

***Index Terms*—Edge AI, on-device LLMs, GGUF, memory estimation, KV cache, grouped-query attention, llama.cpp**

I. Introduction

Quantized language models now run on laptops and small edge servers through llama.cpp [1] and the tools built on it, nearly all of which load the GGUF container format [2]. A user choosing a checkpoint for a device with 8 GB of memory has to decide whether it fits at the intended context length. The runtime reports its allocations, but only after the file is on disk. On a 100 Mbps link a 20 GB checkpoint takes about half an hour to arrive.

The GGUF header already holds the quantities that set most of the memory budget: tensor sizes through the file length, and the attention geometry that sizes the KV cache. The question this paper answers is empirical. How accurate is a header-only estimate against measured peak resident memory, which of its terms matter, and where does it fail? Header-only inspection is not new. GPUStack's gguf-parser [3] and the `@huggingface/gguf` library [4] both read remote GGUF metadata without downloading weights, and gguf-parser and Kolosal's model-memory-calculator [5] also estimate llama.cpp memory. Our contribution is the measured account that, to our knowledge, these tools have not published:

1) An additive estimator whose KV term uses the stored head widths, the cache type's block size and, for Gemma-2, the windowed layers. Its predicted KV cache matches llama.cpp's own allocation log in all 18 configurations we checked (§III).
2) Out-of-sample accuracy on 36 configurations with leave-one-model-out (LOMO) calibration, a worst-case under-prediction and the safety margin it implies, an ablation, and a direct comparison with gguf-parser (§IV).
3) A filled-context and memory-mapping replication. It shows that the short-prompt protocol of our submitted version understated peak memory at long context, and it supplies a header-derived correction (§V).

II. Related Work

Weight quantization (GPTQ [11], AWQ [12]) sets the file size, which is the largest term at short context. At long context the KV cache dominates, and grouped-query attention (GQA) [7] and multi-query attention (MQA) [8] shrink it by the ratio of query heads to KV heads. Gemma-2 alternates full and sliding-window attention layers [9], so part of its cache stops growing with context. PagedAttention [10] manages the cache dynamically for batched serving, which we do not model.

Several memory calculators exist. The Hugging Face model memory calculator [6] multiplies parameter count by bytes per parameter and has no KV term. llama.cpp prints exact allocations once a model is loaded. gguf-parser [3] reads a remote header, accounts for KV heads, cache types and batch sizes, and reports RAM and VRAM estimates with many runtime options. The `@huggingface/gguf` library [4] parses remote metadata but does not estimate memory. Kolosal's model-memory-calculator [5] also reads the header by range request. It sizes the KV cache as hidden size times layers times context, scaled by the ratio of KV heads to heads, and adds 0.02 GB per billion parameters plus 0.15 GB, which it describes as a fit to llama.cpp deployments. That cache formula assumes a head width of $d_{\text{model}}/n_{\text{heads}}$, which is wrong for Gemma-2 (§III), and the tool reports no comparison with measured peaks. We therefore do not claim a capability these tools lack. We compare against gguf-parser directly in §IV and report how close a minimal, calibrated form comes to measured peaks.

III. Estimator

A. Memory decomposition

For one sequence on one backend we write peak resident memory as

$$M = M_{\mathrm{w}} + M_{\mathrm{KV}}(C) + M_{\mathrm{rt}} + M_{\mathrm{att}}(C), \tag{1}$$

where C is the context length in tokens. M_{w} is the sum of file sizes over all shards, obtained from HTTP HEAD requests. With `-no-mmap`, llama.cpp copies the tensor data into memory, so this term is close to exact.

The KV term counts the cells allocated in each layer:

$$M_{\mathrm{KV}}(C) = b(q)\, n_{\mathrm{kv}}\, (d_k + d_v) \sum_{\ell=1}^{L} c_\ell(C). \tag{2}$$

Here n_{kv} is the `head_count_kv` field (equal to the query-head count for multi-head attention and 1 for MQA). d_k and d_v come from `key_length` and `value_length` when present, and from $d_{\mathrm{model}}/n_{\mathrm{heads}}$ otherwise. The distinction matters for Gemma-2-2B, whose head width is 256, not $2304/8 = 288$. The factor $b(q)$ is the storage cost per element of cache type q, including the block scale: 2 bytes for FP16, $34/32 = 1.0625$ for Q8_0, $22/32$ for Q5_0 and $18/32$ for Q4_0. Ordinary layers hold $c_\ell = C$ cells. Gemma-2's windowed layers (every second layer, window $W = 4096$ read from `sliding_window`) hold $\min(C, W + u)$ cells in llama.cpp, where $u = 512$ is the physical batch size. The alternating pattern is architecture knowledge keyed on `general.architecture`; it is not stored in the header.

The runtime term $M_{\mathrm{rt}} = \alpha P + \beta$ absorbs the process image, buffers and allocator slack. We fit α and β by least squares on measurements from the target backend. On our CPU host the fitted α is 0.004 GiB per billion parameters, so the term is effectively the constant $\beta \approx 0.078$ GiB. The parameter count P was absent from all six headers, so we derive it from the file size at 4.65 bits per weight; through α it affects predictions by under 0.02 GiB.

The last term appears only when the context is actually filled (§V). Without flash attention, llama.cpp materializes a 32-bit score matrix of $n_{\mathrm{heads}} \times C \times u$ entries for each batch, and we model its resident share as

$$M_{\mathrm{att}}(C) = \gamma \cdot 4\, u\, n_{\mathrm{heads}}\, C, \tag{3}$$

with γ calibrated like β. A short prompt leaves most of this buffer untouched, so $M_{\mathrm{att}} = 0$ in that regime.

Worked example. For Qwen2-1.5B with an FP16 cache at $C = 8192$: $M_{\mathrm{w}} = 0.918$ GiB; Eq. (2) gives $2 \cdot 2 \cdot 256 \cdot 28 \cdot 8192$ bytes $= 0.219$ GiB; $M_{\mathrm{rt}} = 0.085$ GiB. The estimate of 1.222 GiB compares with 1.233 GiB measured under a short prompt. With a filled context, Experiment 2's coefficients give $M_{\mathrm{att}} = 1.40 \times 0.1875 = 0.263$ GiB and $\beta = 0.078$ GiB, for a prediction of 1.478 GiB against 1.487 GiB measured.

B. What the header read retrieves

The submitted version requested a fixed 256 KB range and stated that it held the whole metadata block. That statement was wrong. We parsed every byte of the six files. The full key–value metadata ends between 0.73 MB (Phi-3) and 7.8 MB (LLaMA-3.2) into the file, mostly because of tokenizer arrays, so every 256 KB request was cut off inside those arrays. The estimator never needs them. In all six files the fields it uses (block count, head counts, head widths, embedding width, sliding window) end within the first 993 bytes, and the length prefix of the token array, which gives the vocabulary size, ends by byte 1,236. The parser therefore issues a 4 KB range request and parses keys in order. If a required key is still missing at the end of the buffer, it doubles the range and continues. It skips array bodies by their declared length and stops once every required field has been read. Sharded checkpoints add one HEAD request per shard. Run against the six files hosted on Hugging Face, the parser needed one 4 KB request per model (0.2–0.5 s each), against 0.37–2.23 GiB of weights.

IV. Experiment 1: Accuracy on the Submitted Grid

A. Setup

The host has two vCPUs (Intel Xeon, 2.9 GHz), 8 GB of RAM and no GPU. It runs Linux 6.x and a CPU-only llama.cpp build (`b1-050dde5`). We used six Q4_K_M checkpoints with query-to-KV head ratios from 1 to 8: Qwen2-0.5B and Qwen2-1.5B, TinyLlama-1.1B, LLaMA-3.2-1B, Gemma-2-2B and Phi-3-mini (multi-head). Each model ran with FP16 and Q8_0 caches at $C \in \{512, 2048, 8192\}$, giving 36 configurations, with a 200-token prompt, 32 generated tokens, `-no-mmap` and three trials. Peak memory is the largest `VmRSS` of the process tree, sampled every 100 ms. Across trials its standard deviation stayed below 0.001 GiB. Memory is reported in GiB (2^{30} bytes). The submitted version labeled these figures GB.

B. Accuracy, calibration and baselines

Table I separates in-sample from LOMO results. In the LOMO rows, α and β are fitted on five models and tested on the sixth. Two confidence intervals are given: a configuration bootstrap and a cluster bootstrap that resamples whole models, since configurations of one model are not independent. The submitted estimator ($d_{\mathrm{model}}/n_{\mathrm{heads}}$, one byte per Q8_0 element, no window) degrades from 2.05% in sample to 2.77% under LOMO. With the corrections of §III, LOMO MAPE is 2.25%. Per-model LOMO MAPE ranges from 1.03% (Gemma-2) to 5.79% (TinyLlama).

We ran gguf-parser v0.26.4 offline on the same files with matched settings: zero GPU layers, the same cache types and context lengths, flash attention off, and its default batch sizes and platform footprint. With `-no-mmap` it never under-predicted, but it over-predicted by 63% on average and by up to 153% on the smallest models. For a fit/no-fit decision on a constrained device, that error is large enough to turn away models that would run. With mmap it under-predicted 30 of 36 configurations, because our measurement copies the weights

TABLE I
PEAK-MEMORY ERROR ON THE 36-CONFIGURATION GRID (E1). CI: 95% BOOTSTRAP OVER CONFIGURATIONS / OVER MODELS.

Estimator	MAPE (%) [CI]	Worst under	Max APE
Submitted, in-sample	2.05	-4.1%	10.4%
Submitted, LOMO	2.77 [2.1–3.5 / 1.6–3.8]	-4.9%	11.1%
Corrected, in-sample	1.58 [1.1–2.1]	-2.8%	5.4%
Corrected, LOMO	**2.25** [1.7–2.9 / 1.1–3.8]	$\mathbf{-3.5}\%$	6.9%
gguf-parser, `-no-mmap`	63.3 [49–78]	$+2.1\%$	153%
gguf-parser, mmap	45.3 [40–52]	-87.7%	88%
Weights only (file size)	19.2 [16–23]	-58.0%	58%

TABLE II
ABLATION OF THE CORRECTED ESTIMATOR (E1, IN-SAMPLE COEFFICIENTS).

Variant	MAPE (%)
Corrected estimator, all 36	1.58
without M_{rt} ($\alpha = \beta = 0$)	8.61
cache always FP16	4.52
n_{heads} in place of n_{kv} (30 GQA configs)	31.16
same, $C = 8192$ only (10 configs)	67.63

into memory. gguf-parser is built to cover many backends and deployment options, so a single matched setting is a narrow test of it. Still, on this host a two-constant calibration was an order of magnitude closer.

Decision margin. We swept device budgets from 0.40 to 6.00 GiB in 0.01 GiB steps over all 36 LOMO predictions (20,160 budget–configuration pairs). Without a margin the estimator wrongly reports a fit in 37 pairs (0.18%). With a 4% margin, which exceeds the worst under-prediction, it makes no false fit, and 98.9% of all decisions are correct. The remaining errors are conservative rejections close to the budget.

C. Ablation and KV-term checks

Table II removes one term at a time from the corrected estimator. The effective KV-head count carries the most weight. Replacing n_{kv} with n_{heads} raises MAPE from 1.58% to 31.2% on the 30 GQA configurations, and to 67.6% on the ten at $C = 8192$, where the cache is largest. The submitted version also reported an F-test against a parameters-only baseline. That test was not valid, because the baseline is not nested in the estimator, and we have removed it.

The measured growth of memory per token from $C = 512$ to 8192 (FP16) is between 1.00 and 1.07 times the prediction of Eq. (2) for all six models. The two largest ratios, 1.07 for TinyLlama and 1.04 for LLaMA-3.2, fit a compute buffer that grows slowly with context, which M_{rt} does not model. For Gemma-2 the ratio was 0.70 in the submitted version and is 1.03 with the true head width and the window rule. Switching from FP16 to Q8_0 at $C = 8192$ saved between 0.99 and 1.01 times the predicted bytes for all six models.

TABLE III
SHORT-PROMPT VERSUS FILLED-CONTEXT PEAK MEMORY (GIB) AT $C = 8192$, FP16 CACHE, WITH LOMO PREDICTIONS WITH AND WITHOUT M_{att} (E2).

Model	Short	Filled	+%	Pred. w/o	Pred. with
Qwen2-0.5B	0.534	0.850	59	0.541	0.851
Qwen2-1.5B	1.220	1.487	22	1.214	1.478
TinyLlama-1.1B	0.845	1.551	84	0.872	1.587
LLaMA-3.2-1B	1.095	1.802	65	1.079	1.774
Gemma-2-2B	2.312	2.515	9	2.303	2.469
Phi-3-mini	5.307	6.018	13	5.306	6.006

V. EXPERIMENT 2: FILLED CONTEXT AND MEMORY MAPPING

Reviewers pointed out that a 200-token prompt measures the memory reserved for a context, not the peak when that context is full, and that real deployments usually keep llama.cpp's default memory-mapped loading. We repeated the measurements on the same host class with a current llama.cpp build (`339f324`, October 2026) and logged its own weight, KV and compute-buffer sizes. This build requires flash attention for a quantized V cache, so Experiment 2 uses FP16 caches with flash attention off.

Replication. Rerunning the short-prompt grid (18 configurations) reproduced Experiment 1 within 1.9% per configuration (mean 1.0%). The KV size predicted by Eq. (2) equaled llama.cpp's logged allocation in all 18 cases, including Gemma-2's split into 416 MiB of full-attention cache and 234 MiB of windowed cache at $C = 8192$. The naive formula gives 832 MiB. With the reserved-capacity form of Eq. (1) ($M_{\mathrm{att}} = 0$), LOMO MAPE was 1.61% and the worst under-prediction -2.3%.

Filled context. We then filled each context with $C - 64$ prompt tokens of plain English text and generated 16 tokens, at $C = 2048$ and 8192 (12 runs, one trial each). Table III gives the results. At $C = 8192$, peak memory rose by 9–84% over the short prompt. The reserved-capacity estimator, calibrated on short prompts, under-predicted filled runs by up to 43.8%. The resident excess grows with $n_{\mathrm{heads}} \cdot C$, as the attention score buffer of Eq. (3) would. Adding that term with LOMO calibration gave a MAPE of 1.71% (cluster CI 0.74–2.92%) and a worst under-prediction of -2.60%. The fitted γ ranged from 1.38 to 1.44 across folds, and β stayed between 0.062 and 0.082 GiB, close to its short-prompt value. A γ above 1 suggests that more than one score-sized buffer is resident at the peak. We report the term as a calibrated fit, not as a derivation of llama.cpp's allocator.

Memory mapping. With the default mmap loading at $C = 2048$, measured `VmRSS` was 6–62% higher than with `-no-mmap`. The log shows why: with mmap, pages of the mapped file and the repacked CPU copy of the weights are resident at the same time. File-backed pages count in RSS but can be reclaimed under pressure, so RSS overstates what mmap actually costs. We specify the estimator for `-no-mmap` and make no accuracy claim for mmap.

VI. Limitations and Conclusion

All measurements come from one class of two-vCPU x86 host under llama.cpp, with models of at most 3.8 B parameters. We did not measure a GPU, a Jetson-class board or a unified-memory laptop. The submitted version included projected totals for 8 B and 70 B models; we have removed them, along with its statement that WebGPU reports device memory, which it does not. The estimator takes device memory as an input; it does not detect it. The reference implementation is a dependency-free Python module, and a browser front end is future work. Filled-context runs used one trial per configuration, and γ was fitted on 12 runs from six models. Enabling flash attention changes the workspace and would need its own calibration. The window rule is specific to Gemma-2 in llama.cpp. Other hybrid, state-space or mixture-of-experts architectures need architecture tables, and batched serving is out of scope.

Within that scope, three findings hold. The GGUF header holds what a fit check needs in its first 1.3 KB. The effective KV-head count and the exact cache geometry account for most of the accuracy. A single calibrated constant then leaves about 2% error out of sample on reserved capacity. Filling the context is a different regime, and an estimator calibrated on short prompts will under-predict it. Adding an attention-workspace term keeps the filled-context error near 2% on this host. The next step is to repeat both experiments on an accelerator and on a unified-memory edge device.

Reproducibility

The parser, measurement harness, raw measurements for both experiments, llama.cpp logs, gguf-parser outputs, model checksums and analysis scripts are available at https://github.com/venumadhav0583/gguf-header-estimator.

Acknowledgment

The author used an AI assistant to help write analysis scripts and edit the text of this version. The author checked every number against the released measurements and is responsible for the content.

References

[1] G. Gerganov *et al.*, "llama.cpp: LLM inference in C/C++," 2023–2026. [Online]. Available: https://github.com/ggml-org/llama.cpp

[2] ggml contributors, "GGUF file format specification," 2024. [Online]. Available: https://github.com/ggml-org/ggml/blob/master/docs/gguf.md

[3] GPUStack, "gguf-parser-go: Review/check GGUF files and estimate the memory usage," v0.26.4, 2026. [Online]. Available: https://github.com/gpustack/gguf-parser-go

[4] Hugging Face, "@huggingface/gguf: A GGUF parser that works on remotely hosted files," 2024. [Online]. Available: https://github.com/huggingface/huggingface.js/tree/main/packages/gguf

[5] Kolosal AI, "model-memory-calculator: GGUF and LLM VRAM calculator," 2025. [Online]. Available: https://github.com/KolosalAI/model-memory-calculator

[6] Hugging Face, "Model memory calculator," 2023. [Online]. Available: https://huggingface.co/spaces/hf-accelerate/model-memory-usage

[7] J. Ainslie *et al.*, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. EMNLP*, 2023.

[8] N. Shazeer, "Fast transformer decoding: One write-head is all you need," arXiv:1911.02150, 2019.

[9] Gemma Team, "Gemma 2: Improving open language models at a practical size," arXiv:2408.00118, 2024.

[10] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[11] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *Proc. ICLR*, 2023.

[12] J. Lin *et al.*, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. MLSys*, 2024.